

ISSR - AI-Powered Funding Intelligence (FOA Ingestion + Semantic Tagging)

GSoC 2026 Proposal

Organisation: HumanAI

Mentors:

- Andrya Allen (University of Alabama)
- Dr Xinyue Ye (University of Alabama)
- Dr Andrea Underhill (University of Alabama)

Applicant: Pratik Ramchandra Pawar

Contact Information

Name: Pratik Ramchandra Pawar

Email: pratikpawar1565@gmail.com (primary) pratik.22311547@viit.ac.in (school)

Phone: +91 9022722494

GitHub: <https://github.com/PratikPawar1401>

LinkedIn: <https://www.linkedin.com/in/pratik-pawar-78a645275/>

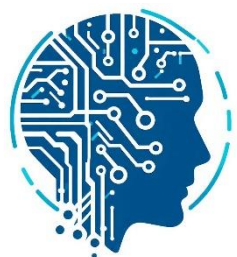
Address: Flat no. 302, G wing, Venkatesh Lake Vista Society, Pune-411046, Maharashtra, India.

Time zone: IST (GMT +5:30)

Total project length: 175 hours



Google Summer of Code



Motivation

I am driven to apply my expertise in document parsing, NLP, and semantic search to solve the fragmented landscape of research funding discovery. During my screening task ([PratikPawar1401/FOA](#)), I built live Grants.gov API pollers and encountered the problem firsthand: missing close dates, truncated descriptions, and FOA text buried in multi-column legacy PDFs that naive parsers returned in the wrong reading order due to column-interleaving. Additionally, API rate limits and inconsistent metadata fields further complicate automated extraction.

My prior projects map directly onto each module of this pipeline. For Atavi Atlas, I built a layout-aware extraction pipeline using transformer-based models to extract structured metadata from complex multi-column records, addressing a column-boundary failure mode that is helpful for parsing FOA-related PDFs. For MediCore ChatBot, I implemented a FAISS-backed Retrieval-Augmented Generation system with semantic tagging over unstructured medical reports, which is architecturally identical to the embedding and tagging layers proposed here. I aim to combine these proven foundations into a scalable, open-source tool that shifts institutional grant scouting from a reactive, manual process into a proactive, structured intelligence pipeline.

Abstract

This project proposes an automated pipeline for ingesting, structuring, and semantically tagging federal Funding Opportunity Announcements (FOAs). The system aggregates opportunities from multiple sources, including the Grants.gov public API and NSF funding pages, combining API retrieval with web scraping to handle both structured and semi-structured announcements. Legacy FOA documents are processed using layout-aware PDF parsing to preserve reading order and extract key program information.

The extracted records are normalised into a standardised JSON schema and enriched using a hybrid semantic tagging framework. Rule-based matching identifies ontology-aligned terms from controlled vocabularies, while sentence embeddings capture semantic relationships for more flexible tagging. The resulting structured dataset enables efficient discovery, search, and future grant-matching workflows.

All components are implemented using open-source tools and run locally, ensuring reproducibility and transparency while creating a foundation for scalable funding intelligence systems.

Phase 0: Community Bonding Period - Research, Setup & Architecture Finalisation (May 1 - May 24)

Total: 15 Hours

Free from academic responsibilities, I will use this time to refine the approach, finalise data standards mappings, and deepen my understanding of the specific models that will be used. This will also be the time to set up all development environments, Docker containers, version control, and core libraries.

Pipeline Architecture Finalisation

During this phase, I will finalise the core architecture of the pipeline, ensuring each component is well-integrated for seamless processing. The pipeline will consist of:

- **Hybrid Ingestion** - Two primary sources: Grants.gov REST API (structured data, public endpoint, no auth required) and Crawlee + Playwright scraping of nsf.gov/funding/opportunities (NSF has no API; solicitations are HTML pages with linked PDFs). NSF RSS feed used as a change-detection signal only, not a content source.
- **Layout-Aware PDF Parsing** - pymupdf4llm processes multi-column FOA PDFs using MuPDF's native layout engine, preserving column reading order and heading hierarchy. pdfplumber supplements table extraction.
- **Semantic Tagging** - Two-layer hybrid: spaCy PhraseMatcher (Layer 1, terminological) followed by all-mpnet-base-v2 cosine-similarity matching (Layer 2, embedding). Mistral-7B-Instruct via Ollama resolves ambiguous conflicts (Layer 3, stretch goal).
- **Vector Search** - FAISS IndexFlatIP indexes all tagged FOA embeddings for efficient retrieval.

Additionally, I will load **GREAT Act Mission Category** labels and **UN SDG** goal titles into a local SQLite ontology store, build synonym expansion tables using WordNet via NLTK, and conduct a structured review of my screening-task code to document all edge cases before development begins.

Phase 1: Weeks 1-3 - Hybrid Ingestion & Layout-Aware Parsing Module (May 25 - June 14)

Total: 36 Hours

Week 1 (May 25 - May 31): Grants.gov API & NSF Change Detection Setup - 12 hrs

- **Environment check:** Run a test against the live Grants.gov API on Day 1 to catch any endpoint changes since bonding. Fix before writing any other logic.
- **Grants.gov API Integration:** Deploy pollers for the search2 and fetchOpportunity endpoints with exponential backoff, cursor-based pagination, and idempotent deduplication by OpportunityID.

- **NSF Change Detection:** Subscribe to the NSF funding RSS feed via feedparser. NSF has no public API - the RSS does not carry full FOA content, it only signals that a new solicitation page has appeared on nsf.gov. Each new RSS entry writes a pending URL to a SQLite queue; the Week 2 scraper drains that queue to fetch the actual content.

Week 2 (June 1-7): NSF Scraper & Normalization - 12 hrs

- **NSF Scraper:** Crawler + Playwright scrapes nsf.gov/funding/opportunities, the actual page where NSF publishes solicitations. The scraper drains the SQLite queue populated by the Week 1 RSS detector, fetches each solicitation page, extracts the HTML content and any linked PDFs, and writes a raw record to the normalisation pipeline. Per-domain scraping rules are stored in YAML for easy extension to additional agency pages.
- **Data Normalization:** Build the canonical raw record schema harmonising payload dates to ISO 8601 format, decoding HTML entities, and normalising whitespace. Both Grants.gov API records and NSF scraped records write to the same schema.

Week 3 (June 8-14): pymupdf4llm PDF Parser Integration - 12 hrs

- **PDF Processing:** Implement pymupdf4llm to convert multi-column historical FOA PDFs to column-ordered Markdown using MuPDF's native layout engine - the critical correctness gate preventing interleaved column text that breaks all downstream extraction.
- **Table Supplement:** pdfplumber extracts embedded tables and appends cell values as structured JSON to the normalized record.
- **Validation Test :** Assert correct column reading order and heading extraction on 15 legacy FOA PDFs collected during the screening task.

Deliverable for Ingestion & Parsing: A functional ingestion module that outputs standardized FOA JSON objects from two sources: Grants.gov via REST API, and NSF via Crawler + Playwright scraping of nsf.gov, with RSS-based change detection as the trigger. Covers HTML pages and multi-column PDFs.

Phase 2: Weeks 4-6 - Schema Normalisation & Terminological Ontology Matching (June 15 - July 5)

Total: 36 Hours

Week 4 (June 15-21): Schema Enforcement & Ontology Data Structures - 12 hrs

- **JSON Schema Validator:** Enforce required fields, ISO 8601 date format, numeric award range fields, and schema_version using the jsonschema library.
- **CSV Export:** Pandas generates a parallel CSV from every validated JSON record. Critically, the CSV includes a tag_evidence column containing the exact text snippet that triggered each tag - making the export directly usable by research development

officers at ISSR without any additional tooling. Both outputs include ingestion_date and schema_version for full reproducibility.

- **Ontology SQLite Store:** GREAT Act Mission Category labels and UN SDG goal/target titles loaded into SQLite; synonym expansion table built using WordNet via NLTK.

The semantic tagging ontology will be organised into four primary conceptual categories to ensure consistent and interpretable classification of FOAs:

1. **Research Domains**

These represent the primary subject areas of the funding opportunity. Initial domain labels will be derived from UN Sustainable Development Goals (SDG) themes and extended with additional domain terms commonly used in federal research programs.

2. **Methods / Approaches**

Tags describing research methodologies or analytical techniques mentioned in the FOA (e.g., machine learning, community-based research, field experiments). These will be identified using rule-based pattern matching combined with embedding similarity.

3. **Populations**

Tags describing the target populations or beneficiary groups of the funding opportunity (e.g., rural communities, veterans, students, aging populations). These will be derived from controlled vocabulary lists and contextual extraction.

4. **Sponsor Themes**

High-level policy or mission priorities defined by funding agencies. These will initially be derived from **GREAT Act Mission Categories** and expanded as needed.

All ontology concepts will be stored in a local SQLite ontology store and mapped to FOA text using the hybrid tagging pipeline described in Phase 2 and Phase 3.

Week 5 (June 22 - June 28): spaCy PhraseMatcher - Layer 1 Terminological Tagger - 12 hrs

The first tagging layer uses spaCy PhraseMatcher - the fastest production-grade exact and near-exact pattern matcher for controlled vocabularies. This is the correct tool for ontology matching: high-precision, linguistically normalised, and fast enough for batch processing without GPU.

- **Pattern Construction:** PhraseMatcher patterns built from GREAT Act category labels, WordNet synonym expansions, and UN SDG goal titles.
- **Lemmatization Pipeline:** spaCy en_core_web_lg tokenizer and lemmatizer normalises inflected forms so "funded", "funding", and "fund" all match the same pattern.
- **Hierarchical Propagation:** Matching a child SDG target automatically tags the parent SDG Goal, ensuring hierarchical completeness.
- **CFDA Crosswalk:** Published CFDA numbers assign Mission Categories where terminological matching returns no result.
- **Evaluation:** Precision/recall on 50-FOA hand-labelled development set; all error categories documented (missing synonyms, abbreviations, cross-domain terms).

Week 6 (June 29 - July 5): Field Extraction Refinement + Midterm Submission - 12 hrs

- **Field Extraction:** spaCy sentence segmentation isolates eligibility and program_description sections from full-document text; regex handles structured fields (award ranges, CFDA numbers, dates).
- **Midterm Submission:** Fully functional hybrid ingestion pipeline, normalised FOA dataset (JSON + CSV) and Layer 1 tagger with documented precision and recall metrics. Submitted by the July 10 midterm evaluation deadline.

Midterm Deliverable (submitted by July 10): The fully integrated hybrid ingestion pipeline alongside a baseline semantic tagging engine that successfully applies terminological matching rules against the GREAT Act and UN SDG ontologies, with documented evaluation metrics.

Phase 3: Weeks 7-8 - all-mpnet-base-v2 Embedding Layer & Multi-Layer Integration (July 6-19)

Total: 30 Hours

Week 7 (July 6-12): all-mpnet-base-v2 Sentence Embeddings - 15 hrs

The second tagging layer uses all-mpnet-base-v2 - a general-purpose sentence transformer that consistently tops the MTEB benchmark for semantic similarity across diverse domains. ISSR's grant portfolio spans social sciences, humanities, arts, and public policy - text that falls entirely outside the scientific citation graphs that domain-specific models like SPECTER2 were trained on. Reimers & Gurevych (2019) demonstrate that sentence transformers trained on broad corpora significantly outperform citation-graph models on cross-domain semantic similarity tasks..

- **Model Setup:** Download and cache all-mpnet-base-v2 weights from Hugging Face; benchmark inference latency on development hardware and document in README.
- **Ontology Embeddings:** Pre-compute and persist all-mpnet-base-v2 embeddings for all ontology concept descriptions to a NumPy .npy cache file - computed once at setup, reused every run.
- **FOA Chunking:** Chunk program_description into 384-token segments (all-mpnet-base-v2 optimal input length); embed each chunk independently and aggregate by mean-pooling.
- **Similarity Scoring:** Cosine similarity between each FOA chunk embedding and cached ontology concept embeddings; threshold at 0.75, calibrated on the 50-FOA development set.

Week 8 (July 13-19): Layer Integration, Automated Evidence Logging & Stretch Goal - 15 hrs

- **Layer Integration Logic:** Layer 1 (spaCy) tags take priority as the high-precision anchor. Layer 2 (all-mpnet-base-v2) fills gaps on unmatched text.

- **Automated Evidence Logging:** Every assigned tag automatically captures a provenance metadata field containing three sub-fields: `source_layer` (Layer 1 or Layer 2), `confidence_score` (cosine similarity for Layer 2; 1.0 for exact terminological matches in Layer 1), and `context_snippet` (the specific sentence or 384-token chunk that triggered the match). This is a purely code-based feature - a single dictionary append in the tagging loop - that makes every output fully auditable and interpretable without any human intervention.
- **Configuration:** Cosine threshold tunable via `.env` config without code changes; evidence logging can be toggled per run but is on by default.
- **Stretch** - Mistral-7B-Instruct Disambiguation : Activated only when the top-2 Layer 2 candidates are within 0.05 cosine similarity of each other - genuine ambiguity. Mistral-7B-Instruct runs locally via Ollama (Apache 2.0 licence, no external API, no data leaving the institution). Prompt templates versioned in `prompts/`. LLM calls are expected to cover fewer than 5% of all tags.

Deliverable for Semantic Tagging Module: A fully functional two-layer semantic tagging engine integrating spaCy terminological matching and all-mpnet-base-v2 embedding similarity, with Automated Evidence Logging (provenance metadata: `source_layer`, `confidence_score`, `context_snippet`) attached to every tag, and optional Mistral-7B-Instruct disambiguation for edge cases.

Phase 4: Week 9 - Evaluation & Grant-Matching Foundation (July 20-26)

Total: 15 Hours

The project description lists a grant-matching foundation as a core deliverable and vector indexing as a stretch goal. This phase delivers both the required evaluation and the matching foundation within the committed scope, with FAISS indexing explicitly treated as an optional extension if time allows.

Week 9 (July 20-26): Evaluation Dataset & Matching Foundation - 15 hrs

- **Evaluation Dataset Construction:** Curate a 30-50 FOA evaluation set hand-labelled by a single annotator against the GREAT Act and UN SDG ontologies. This is the scope the project description calls for - a small, representative set sufficient to demonstrate tagging consistency and surface systematic errors.
- **Summary Metrics Report:** Compute Precision, Recall, and F1 per ontology category across the evaluation set. Document error categories (synonym gaps, abbreviations, cross-domain terms) in a structured summary table.
- **Embedding Export as Matching Foundation:** Export all all-mpnet-base-v2 FOA embeddings as a NumPy `.npy` file alongside the tagged JSON dataset. This is the documented foundation for future grant-matching integration - any downstream system (FAISS, ChromaDB, Weaviate) can consume this file directly without re-embedding.

- **Matching README:** Write a dedicated MATCHING.md documenting the embedding format, schema, and an annotated code snippet showing how a future integrator would load the embeddings and run cosine similarity against a researcher profile.

Deliverable for Evaluation & Matching Foundation: A 30-50 FOA evaluation set with Precision/Recall/F1 summary metrics, plus an exported embedding file and integration guide that constitutes a documented, ready-to-use foundation for future grant-matching systems.

Stretch Goal - FAISS Vector Index & CLI Search (if Phase 1-4 complete early)

If the core deliverables above are completed with time remaining, I will implement a working FAISS IndexFlatIP vector store as an explicit stretch goal. This is the official project stretch goal and is not on the critical path.

- Index all all-mpnet-base-v2 FOA embeddings in FAISS IndexFlatIP (exact cosine, zero approximation error at prototype scale).
- SQLite metadata store maps FAISS integer IDs to foa_id, title, agency, close_date, matched_tags, and source_url.
- CLI: `python search.py --profile "computational social science, housing policy" --k 10` returns ranked FOAs with similarity scores, matched tags, and source URLs.

Note: This stretch goal is only attempted after all six core deliverables are confirmed complete.

Phase 5: Weeks 10-12 - Testing, Refinement & Containerisation (July 27 - August 17)

Total: 35 Hours

This phase uses the 15 hours freed by correctly scoping the grant-matching foundation (Phase 4) as a buffer for end-to-end testing and refinement. Extra time here directly protects the quality of all six core deliverables.

Week 10 (July 27 - August 2): End-to-End Pipeline Integration Testing - 15 hrs

- **Full pipeline run:** ingest -> extract -> normalise -> tag -> export JSON + CSV, end-to-end on a 50-FOA test corpus covering both ingestion sources (Grants.gov API and NSF scraper) and both document formats (HTML and PDF).
- **Schema validation:** confirm every output record passes the JSON Schema validator; log and fix any field-level failures surfaced by real data.
- **Tagging consistency check:** compare Layer 1 and Layer 2 tag assignments on the 30-50 evaluation set built in Phase 4; identify and fix any systematic errors (missed synonyms, threshold mismatches, propagation bugs).
- **Threshold tuning:** Grid search over 0.70-0.80 all-mpnet-base-v2 cosine threshold on the evaluation set to find the F1-optimal value.

Week 11 (August 3-9): Evaluation Report & Iterative Refinement - 10 hrs

- **Evaluation Report:** Using the 30-50 FOA labelled set from Phase 4, compute per-category Precision, Recall, and F1. Crucially, because every tag carries a context_snippet in its provenance field, the error analysis does not rely on guesswork - false positives are diagnosed by reading the exact text that triggered the wrong tag. This turns a standard metrics table into a genuinely actionable report: each error category (synonym gap, abbreviation mismatch, cross-domain confusion) is illustrated with the specific evidence snippet that caused it, and the fix (adding a PhraseMatcher pattern, adjusting a threshold) is traceable to that snippet.
- **Iterative Refinement:** Targeted fixes based on the evidence analysis - adding missing synonym patterns to the PhraseMatcher, adjusting CFDA crosswalk mappings, correcting threshold values. Re-run the evaluation to confirm improvement and update the report.
- **Mentor Review Cycle:** Share the evaluation report and a sample of annotated evidence snippets with the mentor before final submission.

Week 12 (August 10-17): Containerisation & Final Documentation - 10 hrs

- **Dockerisation:** Finalise Dockerfile (python:3.11-slim base) and docker-compose.yml; cold-start reproducibility test on a clean Ubuntu 22.04 image, confirming the full pipeline runs from a single docker-compose up with no manual steps.
- **Dependency Pinning:** requirements.txt with exact pinned versions; .env.example listing all configurable parameters (cosine threshold, source toggles, model paths).
- **README:** Quickstart guide, schema field reference, ontology update instructions, and steps to reproduce the evaluation results.

Deliverable for Testing: Production-ready Dockerised pipeline, a basic evaluation report (Precision/Recall/F1 per category on a 30-50 FOA set), and complete reproducible setup documentation.

Phase 6: Week 13 - Final Report & Handoff (August 18 – August 31)

Week 13: Documentation & Submission

- **Final Documentation:** Architecture diagram, methodology writeup covering all three tagging layers, and evaluation report PDF.
- **Handoff:** Submit the complete codebase to the ISSR / HumanAI repository with a CHANGELOG and full documentation.

Deliverable: Submit the final integrated pipeline with comprehensive documentation and the complete code repository.

Summary of the Timeline

Phase	Timeline	Total Hours	Key Activities / Milestones
0	May 1 - May 24	15 hrs	Community Bonding Period: • Research, setup, Docker scaffold • Ontology data into SQLite, schema • Architecture finalisation with mentor
1	May 25 - Jun 14	36 hrs	Hybrid Ingestion & PDF Parsing: • Grants.gov REST API pollers • NSF scraper (Crawlee + Playwright on nsf.gov) • NSF RSS as a change-detection trigger only • pymupdf4llm PDF parser Deliverable: Ingestion module
2	Jun 15 - Jul 5	36 hrs	Normalisation & Layer 1 Tagging: • JSON Schema validator + CSV export • spaCy PhraseMatcher tagger • CFDA crosswalk, hierarchical propagation Midterm Deliverable: by July 10 deadline
3	Jul 6-19	30 hrs	all-mpnet-base-v2 Embedding Layer: • Ontology concept embeddings cached • Cosine similarity scoring (threshold 0.75) • Two-layer integration + provenance logging • Stretch: Mistral-7B-Instruct disambiguation Deliverable: Full two-layer tagging engine
4	Jul 20-26	15 hrs	Evaluation Dataset & Matching Foundation: • 30-50 FOA labelled evaluation set • Precision / Recall / F1 summary metrics • all-mpnet-base-v2 embeddings exported as .npy • Stretch (if time): FAISS IndexFlatIP + CLI Deliverable: Evaluation set + matching foundation
5	Jul 27 - Aug 17	35 hrs	Testing, Refinement & Containerisation: • End-to-end pipeline integration testing • Evaluation report (P/R/F1 per category) • Iterative refinement from error analysis • Docker finalisation + pinned requirements • README + reproducibility docs Deliverable: Dockerised pipeline + report
6	Aug 18 – Aug 31	-	Final Report: • Architecture diagrams, drawings and workflows document • Final evaluation report

Why Me

1. Relevant Experience

- Built [Atavi Atlas](#), a layout-aware extraction pipeline using transformer-based models to pull structured metadata from complex multi-column records. The column-boundary problem in legacy FOA PDFs is a direct variant of this challenge. My choice of pymupdf4llm over simpler parsers is grounded in that hands-on experience.
- Built [MediCore ChatBot](#) - a FAISS-backed RAG system with semantic tagging over unstructured medical reports. Modules 3 and 4 of this pipeline are architectural variants of that system applied to grant documents. I have debugged FAISS indexing, embedding pipelines, and tag provenance systems in this project.

[My Resume](#)

2. Research Rigour & Competitive Experience

- I have worked under Professor Dr Amar Buchade (Chairperson IEEE Pune), Prof. Prema Kadam and Prof. Dr Parikshit Mahalle at VIIT Pune as a research student on document analysis and NLP projects like Handwritten Character Recognition for Vernacular Languages and Translation, where I gained hands-on experience in deep learning models, especially transformers and deep neural networks.
- Published 2 patents and awaiting confirmation for Handwritten Character Recognition for Vernacular Languages and Translation, and A Collaborative Cloud Coding IDE.
- I have won various competitions in data science and machine learning, which have taught me how to work under pressure and time constraints. I was ranked 284 out of 85k participants for the Amazon ML Challenge.

[LinkedIn Profile](#)

3. Academic Background

- I am pursuing a Bachelor of Technology in Artificial Intelligence and Data Science from VIIT Pune with a CGPA of 9.12. As part of my coursework, I studied Advanced Machine Learning and Fundamentals of Data Science, scoring an Outstanding grade and 10 SGPA in both. Beyond my university courses, I completed Stanford's CS229 Machine Learning MOOC and ZTM's Pytorch Bootcamp, which introduced me to Deep Learning and Reinforcement Learning. I also took hands-on courses from Hugging Face, further strengthening my practical understanding of deep learning in areas like Computer Vision and Natural Language Processing.

[My Current Transcript](#)

4. Open-Source Commitment

- As Part of the Zero to Mastery (ZTM) community and DeepLearning.AI, I have fine-tuned and published models, including Qwen 2.5 and Mistral variants on Hugging Face. This is directly relevant: the all-mpnet-base-v2 integration and the Mistral-7B-Instruct stretch goal in this pipeline use the same fine-tuning and deployment workflow I have already executed.
- Committed to building tools that are modular, documented, and designed for community extension.

Fallback Strategy

The chosen tools are proven open-source components. The primary architecture is highly reliable. However, the fragmented nature of federal data requires robust safeguards:

1. Fallback for PDF Parsing (pymupdf4llm)

- Primary: pymupdf4llm for layout-aware column-ordered text extraction.
- Fallback: If a specific PDF uses non-standard encoding or an encrypted structure that pymupdf4llm cannot open, pdfminer.six provides a byte-stream fallback with custom column-detection heuristics - a pattern I implemented in the Atavi Atlas project. The normalised output format is identical, so all downstream modules are unaffected.

2. Fallback for Web Scraping (Crawlee + Playwright)

- Primary: Crawlee + Playwright for JavaScript-rendered agency pages.
- Fallback: If Crawlee encounters environment compatibility issues in the Docker container, the fallback is Scrapy + Selenium - the stack I used successfully in the screening task. Both write to the same canonical raw record schema, so the normalisation and tagging modules require no changes.

3. Fallback for Embedding Model Availability

- Primary: The embedding model **all-mpnet-base-v2** will be downloaded from Hugging Face during the first pipeline run or during Docker image build.
- Fallback: If the execution environment restricts external downloads, the pipeline supports loading the model from a **pre-cached local model directory** mounted into the container. This allows institutions to provide model weights through internal artifact storage without modifying the architecture.

4. Fallback for LLM Disambiguation (Stretch Goal Only)

- Primary: Ambiguous tag conflicts are resolved using **Mistral-7B-Instruct** running locally via Ollama
- Fallback:
If the Ollama runtime cannot be provisioned in the deployment environment, the pipeline degrades gracefully. Ambiguous tags are not forcibly resolved; instead, they are flagged with "low_confidence": true in the output JSON.
- Optional Development Fallback: For experimentation or development environments, the disambiguation step can optionally call an external LLM API such as the **Gemini API**, provided that appropriate credentials are available. This step is not required for the core pipeline and is disabled by default to preserve full offline reproducibility.

Declaration & Alignment

I am applying exclusively to HumanAI for GSoC 2026. My confidence in this project stems directly from my prior RAG and document extraction builds (Atavi Atlas & MediCore ChatBot), combined with the deep dive into the screening task.

Beyond technical skills, I'm driven to explore how AI can bridge technology and culture.

My previous projects in different domains (Healthcare, EdTech, Geospatial) have taught me to solve critical real-world problems—now I want to apply that rigour in other domains (as a student, I want to explore!). This project is my gateway to merging code with cultural impact, and I'm fully committed to delivering it. The timeline provided above is just an estimate and may shift as the project progresses. However, I'm committed to sticking to it or even surpassing it and will refine it further during the pre-GSoC and community bonding phase. I have no other summer commitments, so I can dedicate around 25-30 hours per week. In the final month, when my college resumes, my availability will drop to about 20 hours per week. To account for this, I plan to front-load a significant portion of the work earlier in the timeline. Each week, my time will be distributed based on workload across planning, learning, coding, documentation, and testing. Documentation will be an ongoing process, integrated seamlessly with development.

Post-GSoC Plan

If time permits, I will do my best to contribute. I will focus on maintaining and enhancing the ingestion pipeline and tagging modules, ensuring they stay robust and up to date. This includes regular updates, bug fixes, and feature improvements based on community feedback from Research development teams, to continuously support the FOA pipeline and dataset expansion.
